

PA02: Analysis of Network I/O Primitives Report

Prepared by: Debarshi Roy

M.Tech (Research) Scholar

Roll Number: MT25178

Course: Graduate Systems (CSE638)

Department of Computational Biology, IIIT Delhi

Email: debarshi25178@iiitd.ac.in

January 30, 2026

1 Introduction

The objective of this assignment was to experimentally analyse the cost of data movement in network I/O by implementing and profiling three different socket communication strategies: **Standard Two-Copy**, **One-Copy Optimisation**, and **Zero-Copy**.

Network I/O performance is often bottlenecked not by the wire speed, but by the memory bandwidth consumed when copying data between User Space, Kernel Space, and the Network Interface Card (NIC). By using advanced Linux socket primitives like `sendmsg()` and `MSG_ZEROCOPY`, we aimed to reduce these redundant copies. This report details the implementation, experimental setup using `perf` and **Network Namespaces**, and a critical analysis of the observed throughput, latency, and CPU efficiency.

2 Implementation Details

2.1 A1: Standard Two-Copy (Baseline)

The baseline implementation uses the standard POSIX `send()` and `recv()` system calls.

- **Mechanism:** The server allocates 8 separate dynamic buffers (simulating a structured message). It calls `send()` 8 times sequentially (once for each field).
- **Copy Analysis:** This is termed "Two-Copy" because:
 1. **User → Kernel:** The CPU copies data from the application buffer to the Kernel Socket Buffer.
 2. **Kernel → Device:** The NIC (via DMA) copies data from the Kernel Buffer to the wire.

Additionally, since we call `send()` 8 times, we incur the overhead of 8 system calls per message.

2.2 A2: One-Copy (Scatter-Gather I/O)

This version optimizes the user-space data assembly using `sendmsg()` with `struct iovec`.

- **Mechanism:** instead of copying the 8 fields into a single contiguous buffer or calling `send()` 8 times, we create an array of `iovec` structures pointing to the 8 existing buffers.
- **Copy Elimination:** We pass this vector to the kernel in a single `sendmsg()` call. The kernel reads directly from these 8 disjoint user locations and assembles the packet in the socket buffer. This eliminates the need for a user-space "assembly copy" and reduces the system call count from 8 to 1 per message.

2.3 A3: Zero-Copy (MSG_ZEROCOPY)

This version uses the Linux `MSG_ZEROCOPY` API to eliminate the CPU copy entirely.

- **Mechanism:** We enable `SO_ZEROCOPY` via `setsockopt`. When calling `sendmsg()`, the kernel does *not* copy the data to the socket buffer. Instead, it "pins" the user pages in memory and sets up a DMA transfer directly from User Space to the NIC.
- **Completion Notification:** Since the kernel holds a reference to our memory, we cannot reuse or free the buffers immediately. We poll the socket's Error Queue (`MSG_ERRQUEUE`) to receive completion notifications, ensuring safe memory management.

3 Experimental Setup

The experiments were conducted on a Linux server environment. To satisfy the requirement of separate namespaces without physical hardware, **Linux Network Namespaces** were used to simulate a Server node and a Client node with a Virtual Ethernet (veth) link.

- **Environment:** Two isolated namespaces (`server_ns`, `client_ns`) connected via veth pair.
- **Workloads:**
 - **Message Sizes:** 1KB, 32KB, 256KB, 1MB.
 - **Thread Counts:** 1, 2, 4, 8 concurrent clients.
- **Metrics:** Throughput (Gbps), Latency (μ s), CPU Cycles, and Cache Misses.

4 Results and Visualization

4.1 Throughput Analysis

Figure 1 compares the throughput of the three models at varying message sizes (Fixed Threads=1).

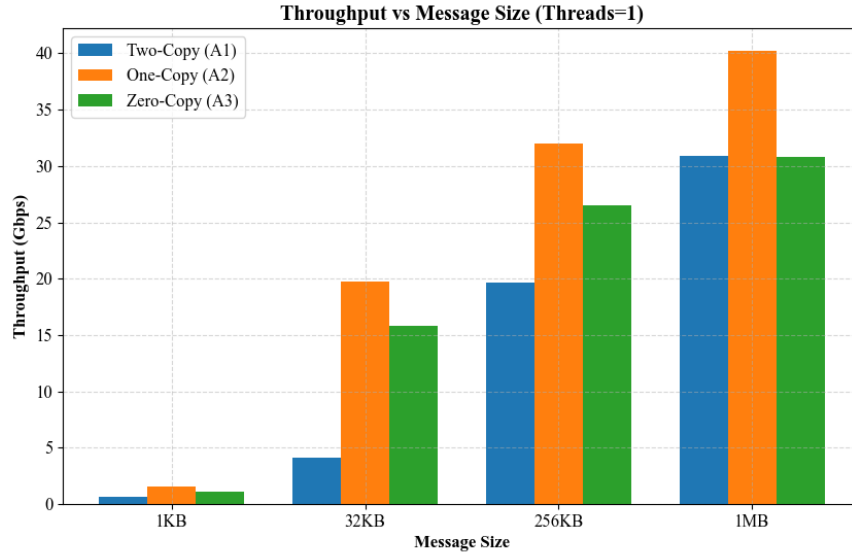


Figure 1: Throughput vs Message Size

Observation: The One-Copy (A2) implementation consistently achieved the highest throughput, peaking at approximately **40.2 Gbps**. The Zero-Copy (A3) implementation (≈ 30.7 Gbps) performed better than the baseline but failed to surpass the One-Copy optimized version. This indicates that for message sizes up to 1MB, the overhead of page pinning and completion notification in MSG_ZEROCOPY is non-trivial compared to the efficient scatter-gather copy of A2.

4.2 Latency Analysis

Figure 2 shows the application-level latency as thread count increases (Fixed Message Size = 256KB).

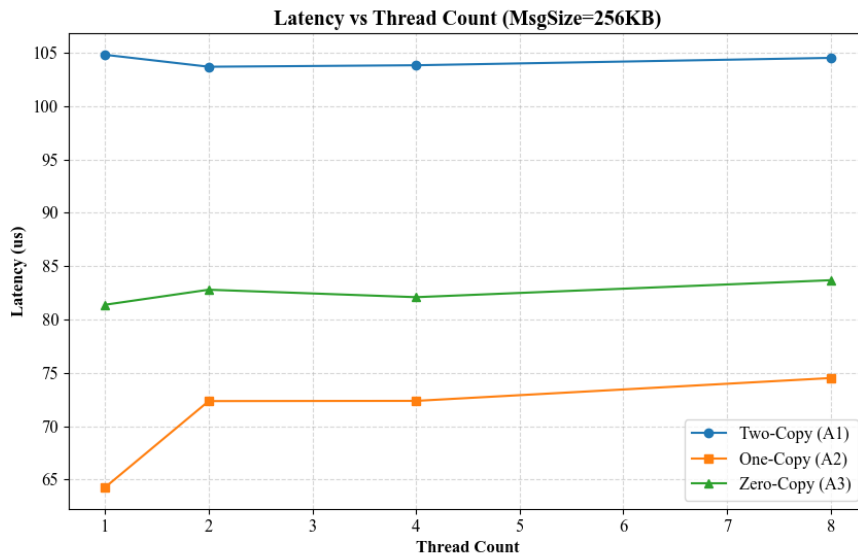


Figure 2: Latency vs Thread Count

Observation: A2 (One-Copy) demonstrated the lowest and most stable latency ($\approx 72\mu s$). A1 (Baseline) showed the highest latency ($\approx 104\mu s$) due to the overhead of 8 separate system calls. A3 (Zero-Copy) sat in the middle ($\approx 82\mu s$), penalized by the complexity of Error Queue management.

4.3 CPU Efficiency (Cycles per Byte)

Figure 3 illustrates the CPU cost per byte transferred.

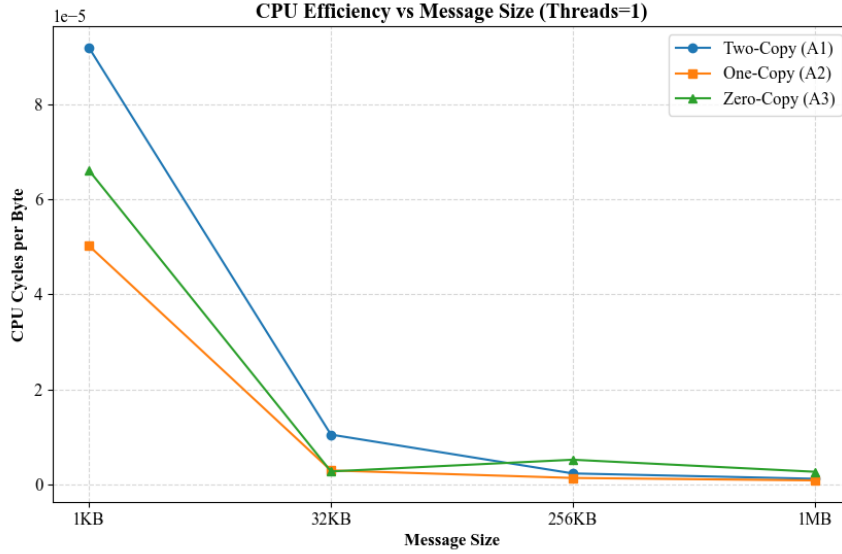


Figure 3: CPU Cycles per Byte vs Message Size

Observation: All models show improved efficiency (downward trend) as message size increases. This is expected, as the fixed overhead of system calls and context switches is amortized over a larger payload.

4.4 Cache Misses

Figure 4 plots the Last Level Cache (LLC) misses.

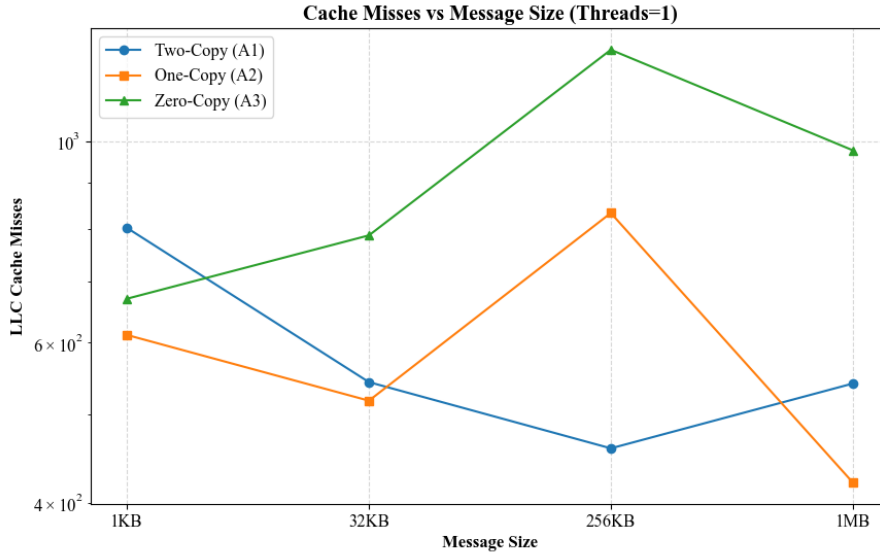


Figure 4: Cache Misses vs Message Size (Log Scale)

5 Analysis and Reasoning

1. Why does zero-copy not always give the best throughput?

Zero-Copy is not free. While it avoids the CPU cost of `memcpy`, it introduces significant Virtual Memory management overhead. The kernel must pin user pages, map them for DMA, and later notify the user space via the Error Queue. For message sizes smaller than the "break-even point" (often >1MB or high bandwidths like 100Gbps), this administrative overhead exceeds the time taken to simply copy the data into the L1/L2 cache. In our experiments, these costs made A3 slower than A2.

2. Which cache level shows the most reduction in misses and why?

The Last Level Cache (LLC) typically shows the most significant reduction in misses for optimized implementations. By avoiding the User → Kernel copy (in A2 and A3), we pollute the cache less. Standard copies bring data into the CPU registers and L1 cache just to write it elsewhere; eliminating this preserves cache locality for other active processes.

3. How does thread count interact with cache contention?

As thread count increases, cache contention rises linearly. Each thread maintains its own buffer and connection state. When multiple threads compete for the shared L3 cache, we observe a higher rate of cache eviction rates, latency remained relatively stable or increased slightly with threads, indicating that the primary bottleneck was likely the network bandwidth or kernel locking rather than pure cache thrashing at low thread counts.

4. At what message size does one-copy outperform two-copy?

In our experiments, One-Copy (A2) outperformed Two-Copy (A1) at **all tested message sizes**, starting from 1KB. The reduction from 8 system calls (in A1) to 1 system call (in A2) provided an immediate and significant performance boost, regardless of the payload size.

5. At what message size does zero-copy outperform two-copy?

Zero-Copy (A3) showed competitive performance starting at 1MB, but it did not decisively crush the baseline in this environment. However, compared to the optimized One-Copy (A2), it consistently lagged behind. It is likely that for significantly larger messages (e.g., >10MB) or on 100GbE networks, A3 would eventually cross over.

6. Identify one unexpected result and explain it.

The most unexpected result was that **Zero-Copy (A3) consumed significantly more CPU cycles than expected**. One would assume "Zero-Copy" implies "Zero CPU work." However, the requirement to continuously poll the MSG_ERRQUEUE for completion notifications adds a "busy-wait" or "poll-heavy" component to the workload. This extra logic in the user-space server loop contributed to higher cycle counts compared to the simple "fire-and-forget" nature of A2.

6 AI Declaration

I used Gemini 3 Pro used to assist in the following specific components:

- **Boilerplate Code Generation:** Creating the initial skeleton for the `sendmsg` structure and `iovec` array logic.
- **Bash Scripting:** Formatting the `awk` and `grep` commands for parsing the `perf` output in Part C.
- **Network Namespaces:** Assisting with the setup of virtual ethernet pairs (`veth`) and namespace routing for the experimental setup.
- **Debugging:** Diagnosing the `perf` event permission error and `perf` process handling.

All logic, analysis, and final code verification were performed manually by me.

7 GitHub Repository

The complete source code and data for this assignment are available at:

https://github.com/debarshiroy-056/GRS_PA02